

Lập trình GIS

GeoPython



GeoPython

GeoPandas

GeoPandas

- ❖ *GeoPandas là một thư viện Python nguồn mở giúp đơn giản hóa việc làm việc với dữ liệu không gian địa lý bằng cách mở rộng các cấu trúc dữ liệu Pandas.*
- ❖ *Thư viện được tích hợp các hoạt động không gian địa lý với giao diện giống như pandas*
- ❖ *GeoPandas Cho phép thao tác các loại hình học như điểm, đường và đa giác,...*



❖ Để sử dụng GeoPandas, ta sẽ thực hiện việc cài đặt thư viện bằng lệnh sau:

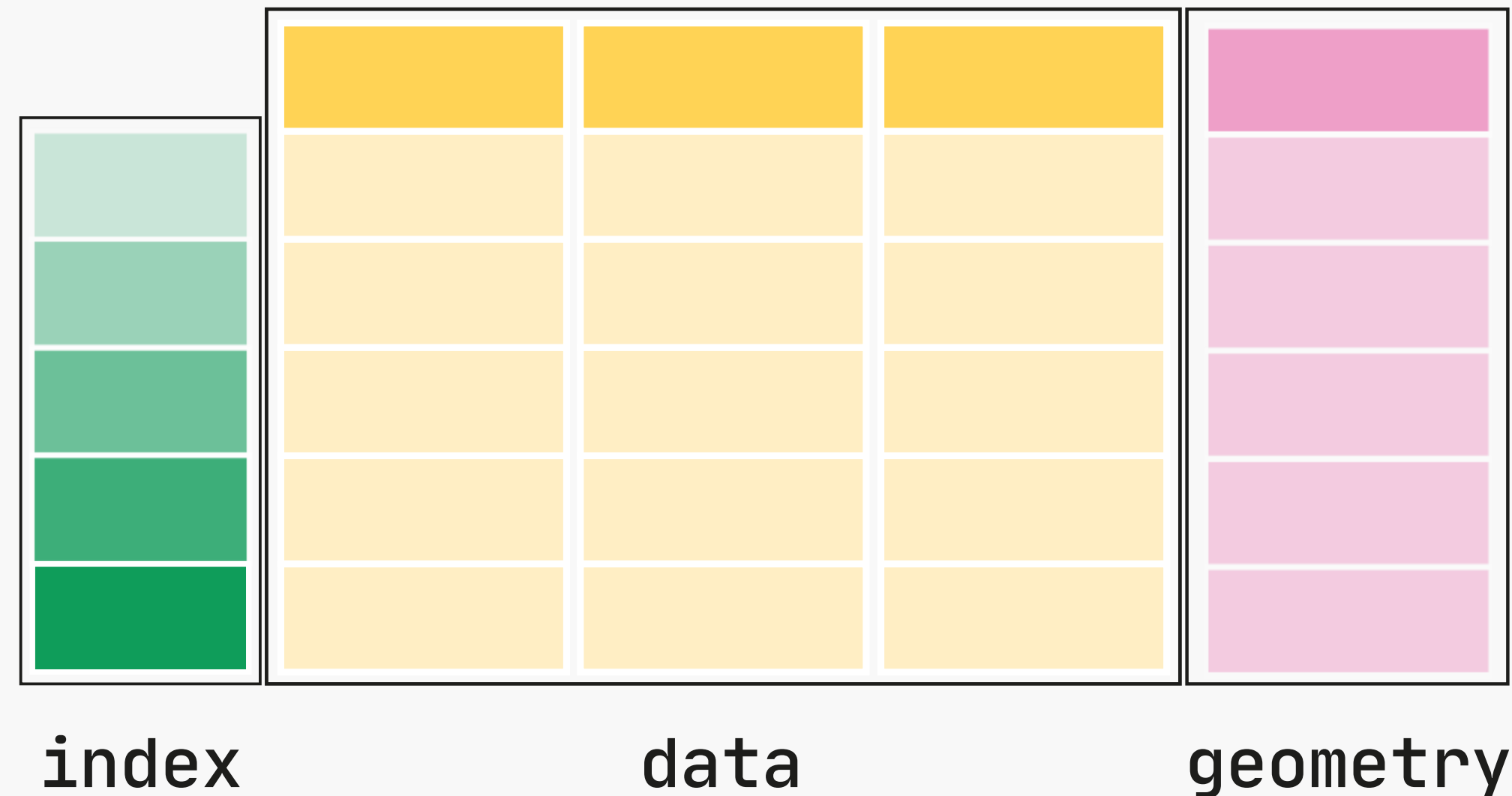
```
pip install geopandas
```

❖ Sau khi đã cài và muốn sử dụng ta sẽ thực hiện import như sau:

```
import geopandas as gpd
```

❖ GeoPandas là thư viện mở rộng của thư viện khoa học dữ liệu phổ biến pandas bằng cách thêm hỗ trợ cho dữ liệu không gian địa lý.

- ❖ Cấu trúc dữ liệu cốt lõi trong GeoPandas là `geopandas.GeoDataFrame`
- ❖ `GeoDataFrame` là một cấu trúc dữ liệu dạng bảng chứa một cột hình học, chứa các hình dạng hình học.
- ❖ Ta có thể tạo `GeoDataFrame` từ danh sách hình học hoặc từ một `DataFrame` pandas.



GeoPandas ► Tạo GeoDataFrame

```
# Khai báo thư viện

import pandas as pd

import geopandas as gpd

import matplotlib.pyplot as plt

# Tạo một GeoDataFrame

data = {

    "City": ["Tokyo", "New York", "London", "Paris"],

    "Latitude": [35.6895, 40.7128, 51.5074, 48.8566],

    "Longitude": [139.6917, -74.0060, -0.1278, 2.3522],

}

df = pd.DataFrame(data)

gdf = gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df.Longitude, df.Latitude))
```

GeoPandas►Tạo GeoDataFrame

```
# Xem thử GeoDataframe vừa tạo
gdf
```

	City	Latitude	Longitude	geometry
0	Tokyo	35.6895	139.6917	POINT (139.6917 35.6895)
1	New York	40.7128	-74.0060	POINT (-74.006 40.7128)
2	London	51.5074	-0.1278	POINT (-0.1278 51.5074)
3	Paris	48.8566	2.3522	POINT (2.3522 48.8566)

GeoPandas►Đọc và ghi dữ liệu không gian địa lý

❖ Đọc file GeoJson

```
url =  
"https://github.com/opengeos/datasets/releases/download/vector/nybb.geojson"  
gdf = gpd.read_file(url)  
gdf.head()
```

	BoroCode	BoroName	Shape_Leng	Shape_Area	geometry
0	5	Staten Island	330470.010332	1.623820e+09	MULTIPOLYGON (((970217.022 145643.332, 970227....
1	4	Queens	896344.047763	3.045213e+09	MULTIPOLYGON (((1029606.077 156073.814, 102957...
2	3	Brooklyn	741080.523166	1.937479e+09	MULTIPOLYGON (((1021176.479 151374.797, 102100...
3	1	Manhattan	359299.096471	6.364715e+08	MULTIPOLYGON (((981219.056 188655.316, 980940....
4	2	Bronx	464392.991824	1.186925e+09	MULTIPOLYGON (((1012821.806 229228.265, 101278...

GeoPandas ► **Đọc và ghi dữ liệu không gian địa lý**

❖ Ghi ra file GeoJson

GeoPandas cũng hỗ trợ lưu dữ liệu không gian địa lý trở lại đĩa. Ví dụ, chúng ta có thể lưu GeoDataFrame dưới dạng tệp GeoJSON mới:

```
output_file = "nyc_boroughs.geojson"
gdf.to_file(output_file, driver="GeoJSON")
print(f"GeoDataFrame has been written to {output_file}")
```



GeoDataFrame has been written to nyc_boroughs.geojson

GeoPandas ► **Đọc và ghi dữ liệu không gian địa lý**

❖ Ghi ra định dạng file khác

```
# Ghi ra file Shapefile
output_file = "nyc_boroughs.shp"
gdf.to_file(output_file)

# Ghi ra file GeoPackage
output_file = "nyc_boroughs.gpkg"
gdf.to_file(output_file, driver="GPKG")
```

GeoPandas ► Các phương thức và truy xuất đơn giản

❖ Tính diện tích: Ta có thể tính diện tích của các quận. GeoPandas tự động tính diện tích của mỗi đa giác:

```
# Set index là cột BoroName
gdf = gdf.set_index("BoroName")
# Tính toán diện tích và thêm cột area vào dataframe
gdf["area"] = gdf.area
gdf
```

	BoroCode	Shape_Leng	Shape_Area	geometry	area
BoroName					
Staten Island	5	330470.010332	1.623820e+09	MULTIPOLYGON (((970217.022 145643.332, 970227....	1.623822e+09
Queens	4	896344.047763	3.045213e+09	MULTIPOLYGON (((1029606.077 156073.814, 102957...	3.045214e+09
Brooklyn	3	741080.523166	1.937479e+09	MULTIPOLYGON (((1021176.479 151374.797, 102100...	1.937478e+09
Manhattan	1	359299.096471	6.364715e+08	MULTIPOLYGON (((981219.056 188655.316, 980940....	6.364712e+08
Bronx	2	464392.991824	1.186925e+09	MULTIPOLYGON (((1012821.806 229228.265, 101278...	1.186926e+09

GeoPandas ► Các phương thức và truy xuất đơn giản

❖ Tính đường biên đa giác và tâm

```
# Để lấy ranh giới (các đường thẳng) và tâm (điểm trung tâm) của mỗi đa giác:

# Tính đường biên của mỗi đa giác

gdf["boundary"] = gdf.boundary

# Tính tâm của mỗi đa giác

gdf["centroid"] = gdf.centroid

gdf[["boundary", "centroid"]]
```

	boundary	centroid
BoroName		
Staten Island	MULTILINESTRING ((970217.022 145643.332, 97022...	POINT (941639.45 150931.991)
Queens	MULTILINESTRING ((1029606.077 156073.814, 1029...	POINT (1034578.078 197116.604)
Brooklyn	MULTILINESTRING ((1021176.479 151374.797, 1021...	POINT (998769.115 174169.761)
Manhattan	MULTILINESTRING ((981219.056 188655.316, 98094...	POINT (993336.965 222451.437)
Bronx	MULTILINESTRING ((1012821.806 229228.265, 1012...	POINT (1021174.79 249937.98)

GeoPandas► Các phương thức và truy xuất đơn giản

❖ Tính khoảng cách

```
# Ta sử dụng tâm Manhattan là tâm tham chiếu
manhattan_centroid = gdf.loc["Manhattan", "centroid"]
# Tính khoảng cách từ mỗi tâm đến tâm tham chiếu Manhattan
gdf["distance_to_manhattan"] = gdf["centroid"].distance(manhattan_centroid)
gdf[["centroid", "distance_to_manhattan"]]
```

	centroid	khoang_cach_den_manhattan
BoroName		
Staten Island	POINT (941639.45 150931.991)	88247.742789
Queens	POINT (1034578.078 197116.604)	48401.272479
Brooklyn	POINT (998769.115 174169.761)	48586.299386
Manhattan	POINT (993336.965 222451.437)	0.000000
Bronx	POINT (1021174.79 249937.98)	39121.024479

GeoPandas ► Các phương thức và truy xuất đơn giản

❖ Tính khoảng cách trung bình

```
#Tính khoảng cách trung bình
```

```
mean_distance = gdf["khoang_cach_den_manhattan"].mean()
```

```
print(f"Trung bình khoảng cách đến Manhattan: {mean_distance}")
```

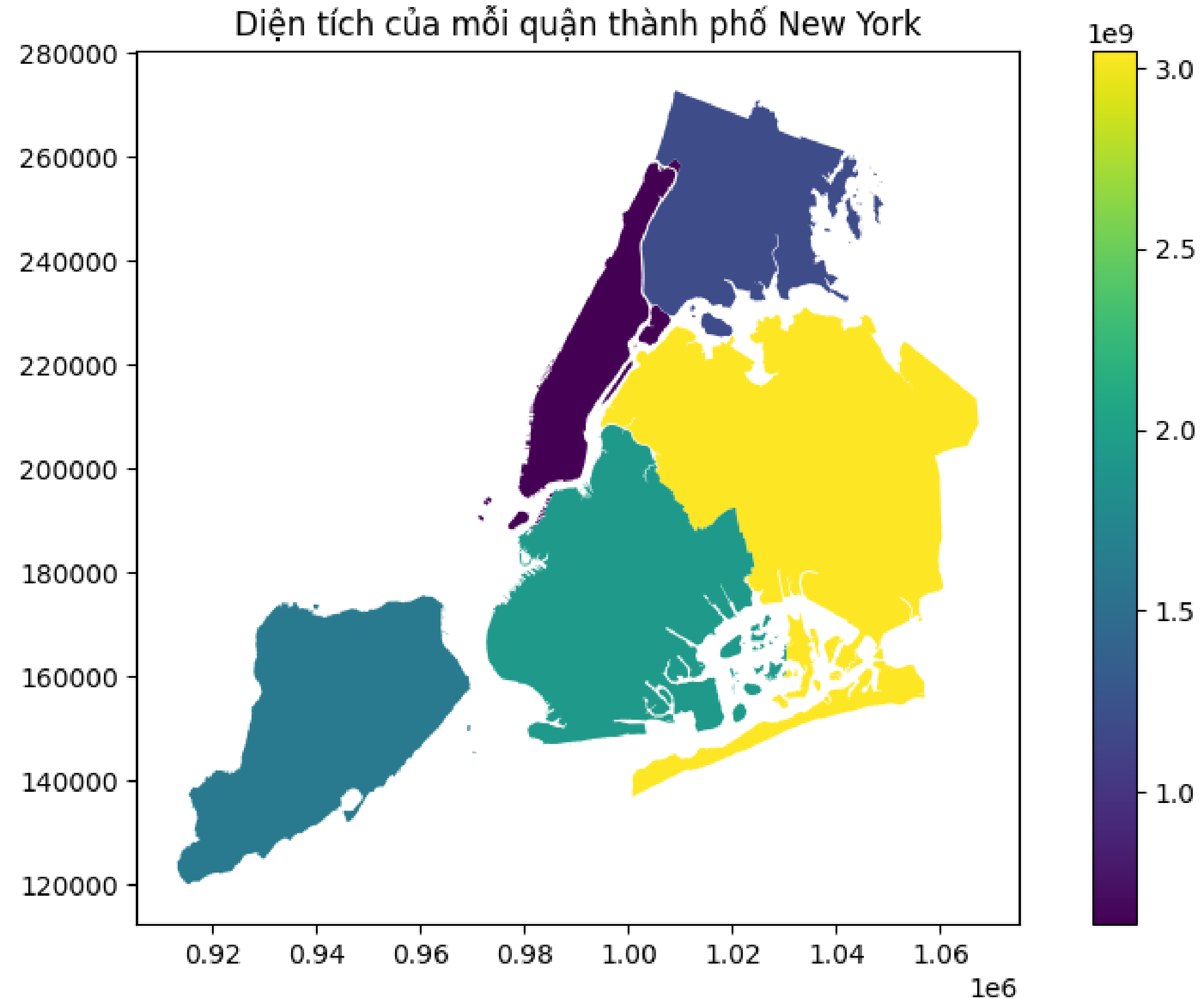
```
Trung bình khoảng cách đến Manhattan: 44871.26782659276
```

GeoPandas ► **Vẽ đồ thị dữ liệu không gian địa lý**

GeoPandas tích hợp với Matplotlib dễ dàng vẽ đồ thị để biểu diễn dữ liệu không gian địa lý.

❖ **Vẽ diện tích của mỗi quận**

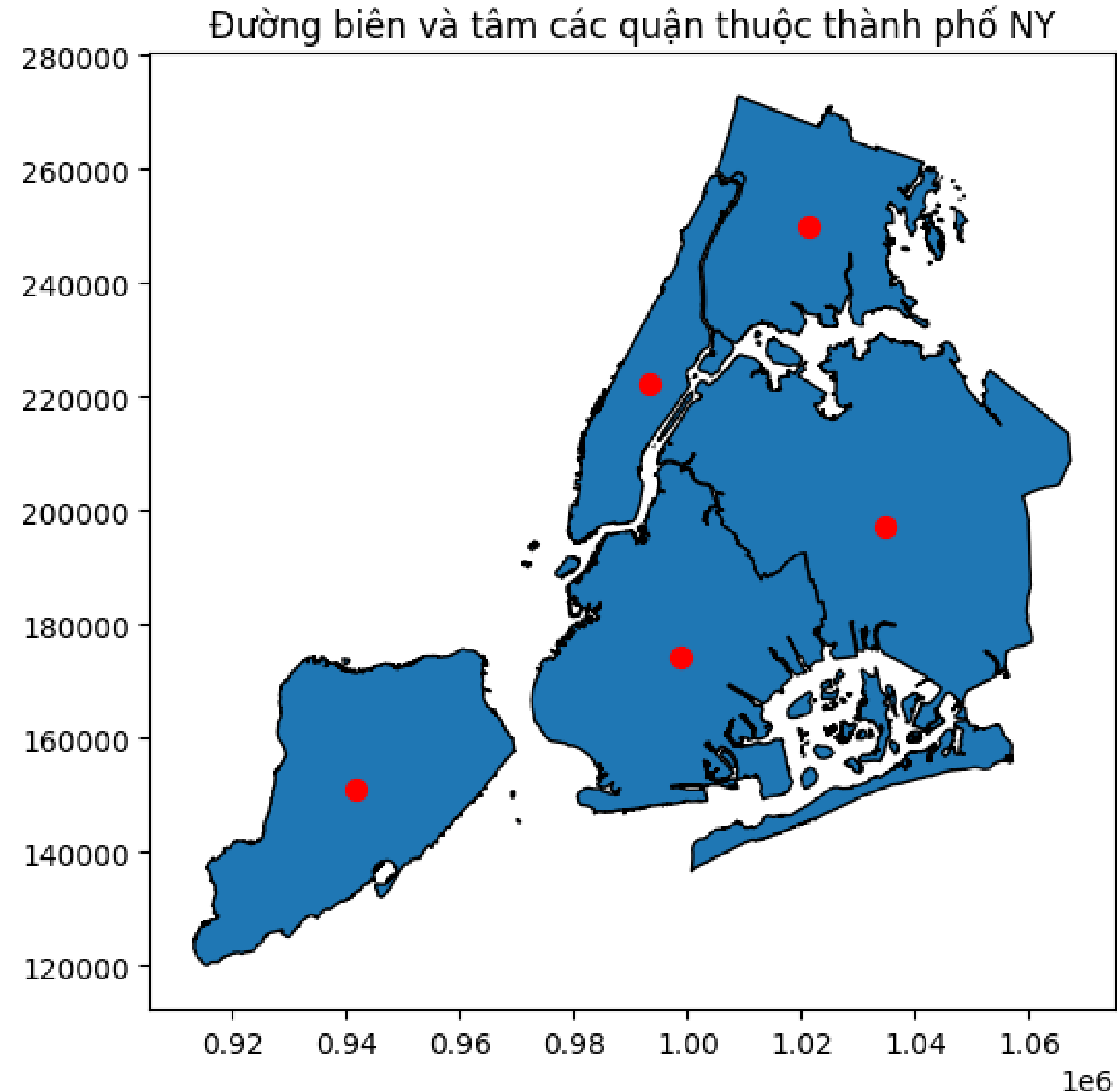
```
gdf.plot("area", legend=True,  
figsize=(10, 6))  
plt.title("Diện tích của mỗi  
quận thành phố New York")  
plt.show()
```



GeoPandas ► Vẽ đồ thị dữ liệu không gian địa lý

❖ Vẽ tâm và đường biên của các quận

```
# Vẽ đường biên và tâm  
ax = gdf["geometry"].plot(figsize=(10,  
6), edgecolor="black")  
gdf["centroid"].plot(ax=ax, color="red",  
markersize=50)  
plt.title("Đường biên và tâm các quận  
thuộc thành phố NY")  
plt.show()
```



GeoPandas ► **Vẽ đồ thị dữ liệu không gian địa lý**

Để tạo một bản đồ tương tác có thể sử dụng `GeoDataFrame.explore()`, và để sử dụng được hàm `explore()` ta cần cài một số thư viện sau:

```
# Cài thư viện folium để sử dụng được bản đồ
pip install folium

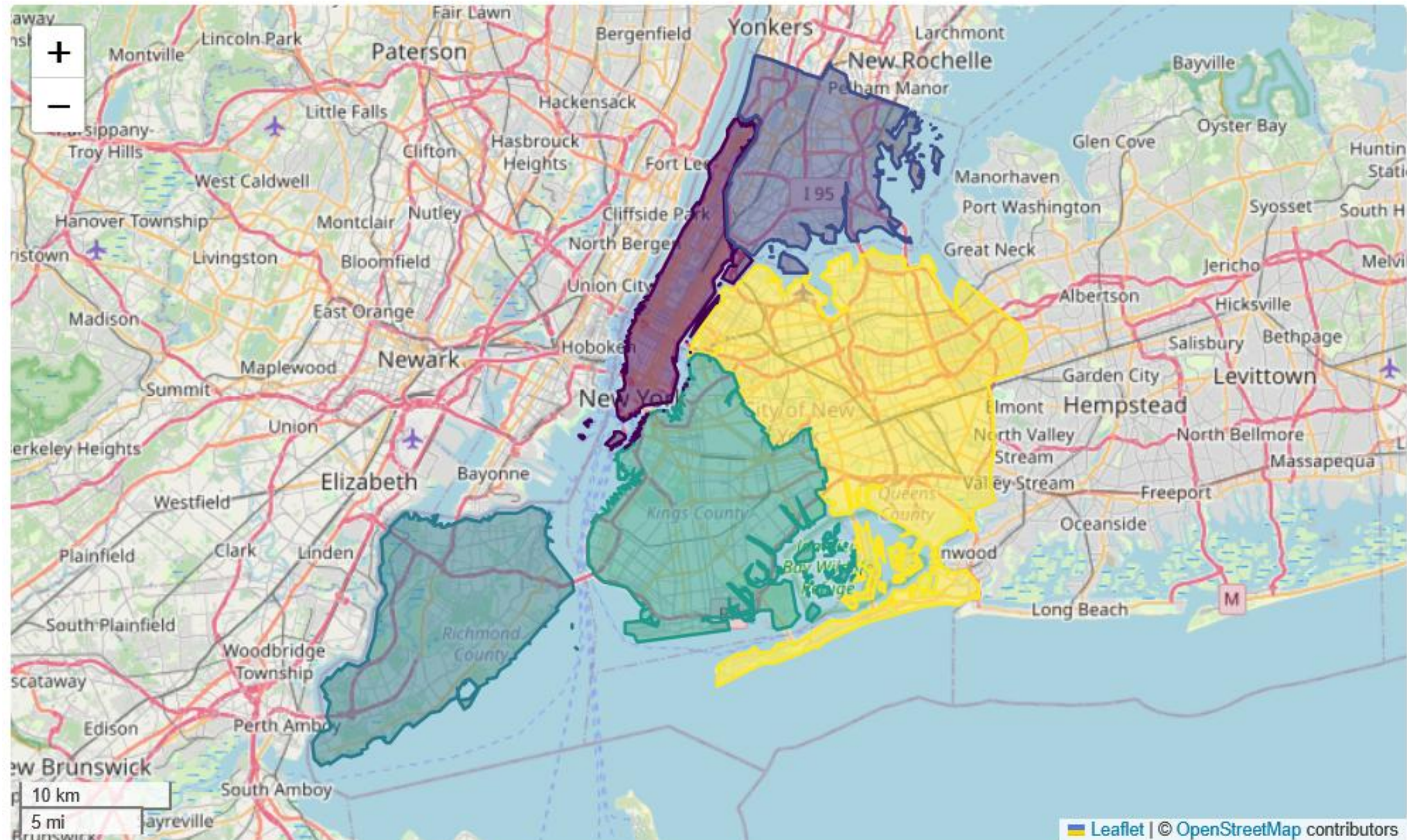
# Cài thư viện mapclassify
pip install mapclassify

# Cài thư viện matplotlib nếu chưa cài trước đó
pip install matplotlib
```

GeoPandas ► **Vẽ đồ thị dữ liệu không gian địa lý**

Giờ ta bắt đầu chạy lệnh sau để tạo bản đồ có thể tương tác

```
gdf.explore("area", legend=False)
```



GeoPython

Leafmap

Leafmap

- ❖ *Leafmap là một thư viện Python được thiết kế để tạo bản đồ tương tác và thực hiện phân tích không gian địa lý với lập trình đơn giản.*
- ❖ *Đặc biệt là trong môi trường Jupyter Notebook. Nó dễ sử dụng cho người mới bắt đầu và cung cấp nhiều chức năng để xử lý dữ liệu không gian địa lý.*



Leafmap ► Cài đặt

❖ Leafmap hỗ trợ nhiều loại vẽ đồ thị, bao gồm folium, ipyleaflet, maplibre, bokeh, pydeck, keplergl và plotly. Vẽ đồ thị mặc định là ipyleaflet, cung cấp các tính năng mở rộng nhất.

❖ Để sử dụng Leafmap, ta sẽ thực hiện việc cài đặt thư viện bằng lệnh sau:

```
pip install Leafmap
```

❖ Sau khi đã cài và muốn sử dụng ta sẽ thực hiện import như sau:

```
import leafmap
```

Leafmap► Khai báo các thư viện cần thiết để thực hiện vẽ 1 số bản đồ trực quan

❖ Để sử dụng Leafmap, ta sẽ khai báo 1 số thư viện sau:

```
import leafmap.foliumap as leafmap
import leafmap.bokehmap as leafmap
import leafmap.maplibregl as leafmap
import leafmap.deck as leafmap
import leafmap.kepler as leafmap
import leafmap.plotlymap as leafmap
```

Leafmap ► Thao tác với Leafmap

❖ Tạo bản đồ tương tác

Leafmap cung cấp giao diện để tạo bản đồ tương tác. Ta có thể tạo bản đồ tương tác cơ bản bằng cách gọi hàm `Map()`. Bản đồ nền mặc định là của *OpenStreetMap*

```
m = leafmap.Map()
```

```
m
```



Leafmap ► Thao tác với Leafmap

❖ Tùy biến bản đồ:

- Ta có thể tùy chỉnh tâm bản đồ, cấp độ zoom và chiều cao.
- Thay đổi tâm bản đồ, zoom là một số nguyên và chiều cao bản đồ tính bằng pixel.
- **Ví dụ: Ta** căn giữa bản đồ tại TP Hồ Chí Minh với vĩ độ là 10, kinh độ là 106 , zoom là 4 và chiều cao là 450 pixel, chiều rộng là 600 pixel

Leafmap ► Thao tác với Leafmap

```
m = leafmap.Map(center=(10, 106), zoom=4, height="450px", width="600px")
```

m



Leafmap ► Thao tác với Leafmap

❖ Thêm hoặc xóa các điều khiển trên bản đồ:

- Theo mặc định, bản đồ bao gồm các điều khiển như zoom, fullscreen, scale, attribution và toolbar.
- Ta có thể chuyển đổi các điều khiển này bằng các tham số như zoom_control, fullscreen_control, scale_control, attribution_control và toolbar_control thành **True** hoặc **False**.
- Ví dụ bên dưới vô hiệu hóa tất cả các control:

```
m = leafmap.Map(  
    zoom_control=False, draw_control=False,  
    scale_control=False, fullscreen_control=False,  
    attribution_control=False, toolbar_control=False, center=(10, 106), zoom=4,  
    height="450px", width="600px")
```

m

Leafmap ► Thao tác với Leafmap

❖ Kết quả



Leafmap ► Thao tác với Leafmap

❖ Thêm control tìm kiếm

- Để thêm control tìm kiếm vào bản đồ, sử dụng phương thức `m.add_search_control()`.
Control tìm kiếm này cho phép người dùng tìm kiếm địa điểm và phóng to đến đó.

```
url =  
https://nominatim.openstreetmap.org/search?format=json&q={s}  
m.add_search_control(url, zoom=10,  
position="topleft")
```



Leafmap ► Thao tác với Leafmap

❖ Làm việc với Layers

- Để có thể truy xuất đến các layer trên bản đồ, ta sẽ sử dụng thuộc tính layers

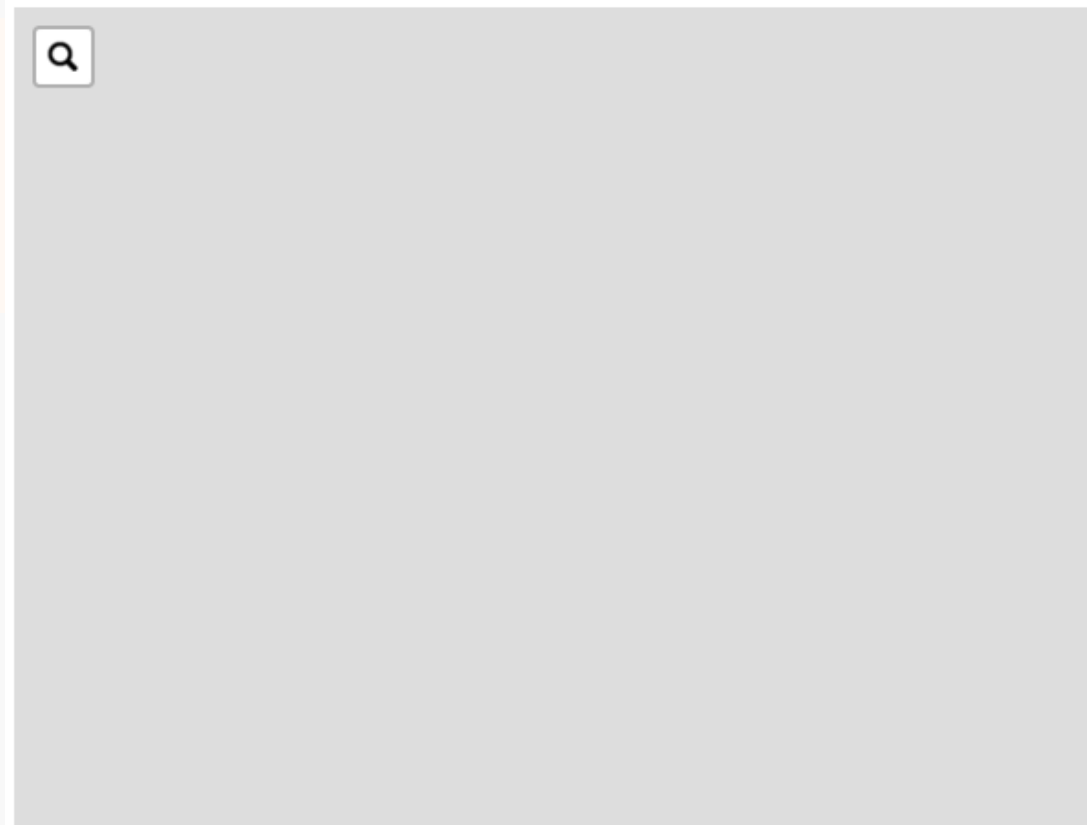
#Để truy xuất thông tin các layer trên bản đồ, ta sử dụng thuộc tính layers

```
m.layers
```

```
(TileLayer(attribution='&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>  
contributors', base=True, max_zoom=19, min_zoom=1, name='OpenStreetMap', options=['attribution', '  
bounds', 'detect_retina', 'max_native_zoom', 'max_zoom', 'min_native_zoom', 'min_zoom', 'no_wrap',  
'tile_size', 'tms', 'zoom_offset'], url='https://tile.openstreetmap.org/{z}/{x}/{y}.png'),)
```

Để remove

```
layerm.remove(m.layers[-1])
```



Leafmap ► Thao tác với Leafmap

❖ Xóa control và layer

- Ta có thể xóa tất cả các control và layer từ bản đồ với phương thức `clear_controls` và `clear_layers`

```
# Trước tiên ta thực hiện tạo một bản đồ
```

```
m = leafmap.Map()
```

```
m
```

```
# Nếu muốn xóa tất cả các control
```

```
m.clear_controls()
```

```
# Nếu muốn xóa tất cả các layer
```

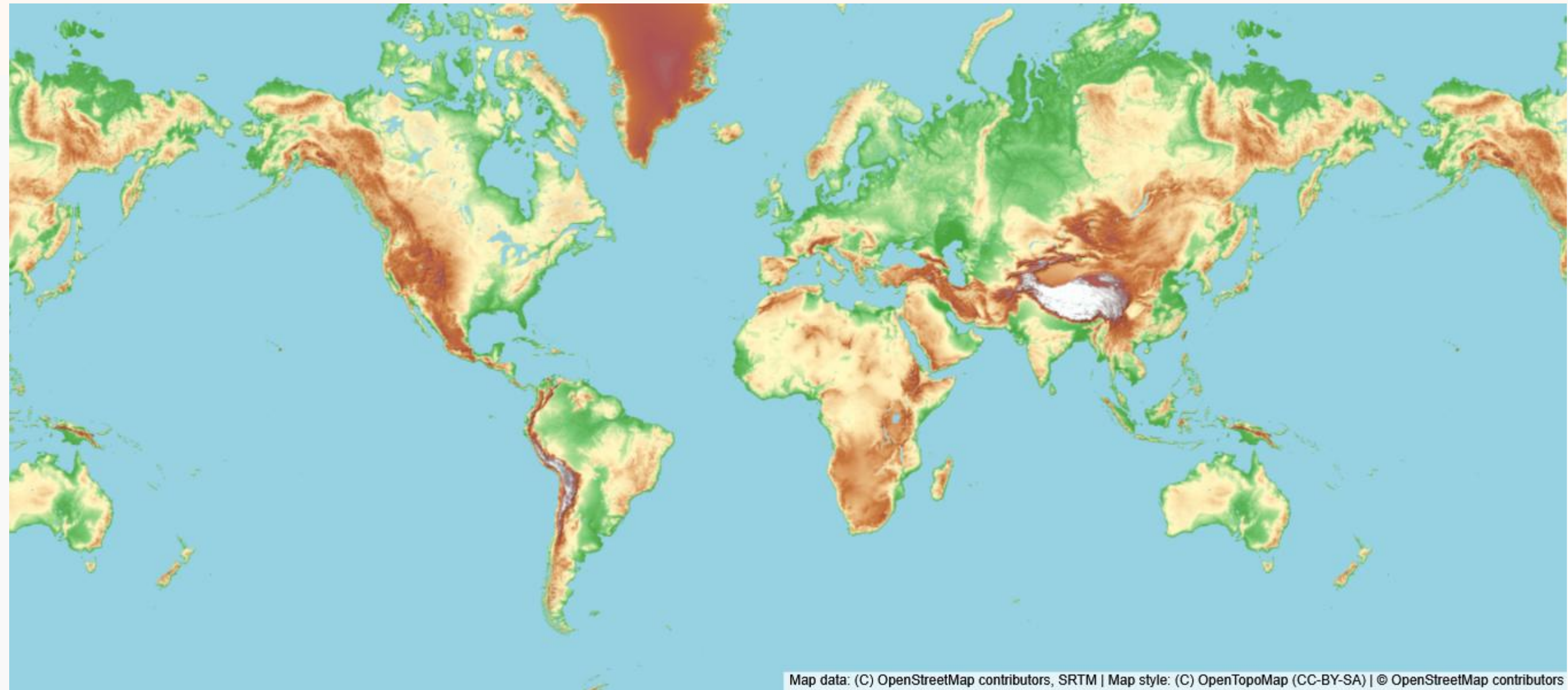
```
m.clear_layers()
```

Leafmap ► Thao tác với Leafmap

❖ Thay đổi bản đồ nền

- Ta có thể dễ dàng thay đổi bản đồ nền của bản đồ. Ví dụ, đoạn mã sau đây thêm bản đồ nền OpenTopoMap:

```
m = leafmap.Map()  
m.add_basemap("OpenTopoMap")  
m
```



Leafmap► Thao tác với Leafmap

❖ Trực quan hóa GeoPandas GeoDataFrames

- Bạn có thể trực tiếp trực quan hóa GeoPandas GeoDataFrames trên bản đồ. Trong ví dụ này, một GeoDataFrame chứa dấu các tòa nhà từ Las Vegas được hiển thị với kiểu dáng tùy chỉnh:

```
import geopandas as gpd

url =
"https://github.com/opengeos/datasets/releases/download/places/las_vegas_buildin
gs.geojson"
gdf = gpd.read_file(url)

gdf.head()
```

	id	height	geometry
0	08b2986b81b34fff0200120c6aabf673	2.539160	POLYGON ((-115.20758 36.11913, -115.20754 36.1...
1	08b2986b81b34fff0200ce30fa61746d	3.209284	POLYGON ((-115.20756 36.11929, -115.20776 36.1...
2	08b2986b81b34fff0200b898b488ab8b	2.691431	POLYGON ((-115.20759 36.11932, -115.20759 36.1...
3	08b2986b81b34fff02004b957b4cd490	2.795901	POLYGON ((-115.20762 36.11953, -115.20775 36.1...
4	08b2986b81869fff020091d66f33d09b	2.649764	POLYGON ((-115.20727 36.11898, -115.20727 36.1...

Leafmap ► Thao tác với Leafmap

❖ Trực quan hóa GeoPandas GeoDataFrames (tiếp theo)

```
gdf.explore()
```

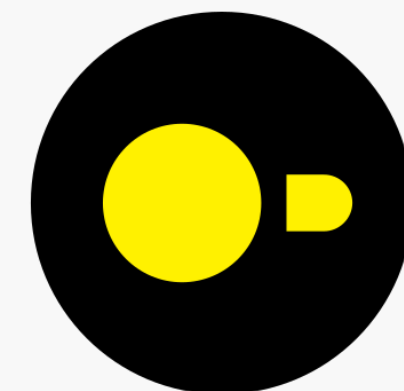


GeoPython

DuckDB

DuckDB

- ❖ *DuckDB là một cơ sở dữ liệu quan hệ mã nguồn mở được thiết kế để làm việc với dữ liệu lớn một cách nhanh chóng và hiệu quả.*
- ❖ *Nó có một số ưu điểm vượt trội, như khả năng xử lý truy vấn SQL nhanh hơn so với nhiều hệ thống cơ sở dữ liệu khác, đặc biệt là trong các tình huống cần phân tích dữ liệu lớn.*
- ❖ *Nó sử dụng Python package duckdb để kết nối với cơ sở dữ liệu DuckDB và nhập dữ liệu từ nhiều định dạng khác nhau, bao gồm CSV, JSON, DataFrame, parquet, GeoJSON, Shapefile, GeoParquet, ...*



DuckDB

DuckDB ► Cài đặt và khai báo sử dụng

❖ Để biết thêm nhiều về DuckDB có thể truy cập vào đường link sau để đọc thêm: ***<https://duckdb.org/docs/>***

❖ Để sử dụng DuckDB, ta sẽ thực hiện việc cài đặt thư viện bằng lệnh sau:

pip install duckdb

❖ Sau khi đã cài và muốn sử dụng ta sẽ thực hiện import như sau:

import duckdb

DuckDB ► Ví dụ

❖ Giờ ta thực hiện tạo một db để thêm dữ liệu vào db

```
# Khai báo thư viện
```

```
import duckdb
```

```
# Kết nối đến cơ sở dữ liệu DuckDB
```

```
con = duckdb.connect(database='ltgis_db.db')
```

```
# Tạo một bảng và nhập dữ liệu
```

```
con.execute("CREATE TABLE ltgis (id INTEGER, name STRING)")
```

```
con.execute("INSERT INTO ltgis VALUES (1, 'Bài học buổi 1')")
```

```
con.execute("INSERT INTO ltgis VALUES (2, 'Bài học buổi 2')")
```

```
con.execute("INSERT INTO ltgis VALUES (3, 'Bài học buổi 3')")
```

DuckDB ► Ví dụ

❖ Thực hiện truy vấn dữ liệu sau khi tạo để xem kết quả

```
# Thực hiện truy vấn SQL và lấy kết quả thành DataFrame
```

```
result = con.execute("SELECT * FROM ltgis").fetchdf()
```

```
[7] result
```

```
✓ 0.0s
```

```
...
```

	id	name
0	1	Bài học buổi 1
1	2	Bài học buổi 2
2	3	Bài học buổi 3

DuckDB ► Ví dụ

❖ Chuyển dữ liệu từ DuckDB sang Pandas Dataframe

```
# Khai báo thư viện
```

```
import pandas as pd
```

```
# Chuyển kết quả từ DuckDB thành DataFrame của Pandas
```

```
df = pd.DataFrame(result, columns=result.columns)
```

```
[6] ✓ 0.0s
```

```
...
```

	id	name
0	1	Bài học buổi 1
1	2	Bài học buổi 2
2	3	Bài học buổi 3

DuckDB► Ví dụ

❖ Ngoài ra ta có thể sử dụng DuckDB để dữ liệu từ file CSV, Json, GeoJson, ...

```
# Khai báo thư viện
import duckdb

# Thực hiện nối kết đến DuckDB
con = duckdb.connect()

# Đọc từ file csv
con.read_csv("cities.csv")
```

id int64	name varchar	state_id int64	state_code varchar	state_name varchar	country_id int64	country_code varchar	country_name varchar	latitude double
52	Ashkāsham	3901	BDS	Badakhshan	1	AF	Afghanistan	36.68333
68	Fayzabad	3901	BDS	Badakhshan	1	AF	Afghanistan	37.11664
78	Jurm	3901	BDS	Badakhshan	1	AF	Afghanistan	36.86477
84	Khandūd	3901	BDS	Badakhshan	1	AF	Afghanistan	36.95127

DuckDB► Ví dụ

❖ Đọc dữ liệu từ file CSV và chuyển dữ liệu sang Pandas Dataframe

```
# Khai báo thư viện

import duckdb

# Thực hiện nối kết đến DuckDB

con = duckdb.connect()

# Đọc từ file csv

df = con.sql("SELECT * FROM 'cities.csv'").df()
```

df.head()
✓ 0.0s

	id	name	state_id	state_code	state_name	country_id	country_code	country_name	latitude	longitude	wikiDataId
0	52	Ashkāsham	3901	BDS	Badakhshan	1	AF	Afghanistan	36.68333	71.53333	Q4805192
1	68	Fayzabad	3901	BDS	Badakhshan	1	AF	Afghanistan	37.11664	70.58002	Q156558
2	78	Jurm	3901	BDS	Badakhshan	1	AF	Afghanistan	36.86477	70.83421	Q10308323
3	84	Khandūd	3901	BDS	Badakhshan	1	AF	Afghanistan	36.95127	72.31800	Q3290334
4	115	Rāghistān	3901	BDS	Badakhshan	1	AF	Afghanistan	37.66079	70.67346	Q2670909